Indexação e Estruturas de Índices

O que são índices

- São estruturas de dados que organizam referências as localizações dos dados reais das tabelas.
- Por analogia, os índices nos banco de dados são similares aos índices apresentados em livros

- **Importância:**

- Lembre-se, que conforme o banco de dados cresce, em um determinado momento o desempenho das consultas tende a cair, não sendo mais satisfatório
- Os índices são utilizados para melhorar o tempo de execução das consultas, sendo recursos bastante utilizados para a resolução desse problema

Tabelas (padrão)

- Tabela é um conjunto desordenado de **tuplas** exclusivas.

- Por padrão, os dados são armazenados sem uma ordem particular, isto significa que:

- A adição de novos registo não necessita de uma reorganização dos dados existentes → melhor desempenho na inserção de dados

- A recuperação de informações é feita por busca por cada registro presente na tabela → maior tempo para a obtenção da informação.

Tabelas organizadas por índice

- Com os índices temos a nossa disposição uma estrutura adicional de organização dos dados de todas as colunas das tabelas, melhorando a pesquisa de informações

- Mas precisamos ter cuidado com a quantidade de índices por tabela, isso devido a sobrecarga durante as operações DML (INSERT, UPDATE e DELETE), pois quando cada uma das operações realizadas, é necessária a atualização dos índices

```
CREATE INDEX nome_index ON nome_tabela (nome_coluna);
```

```
CREATE INDEX idx_produtos_nome ON produtos(nome);
```

Tipos de índices

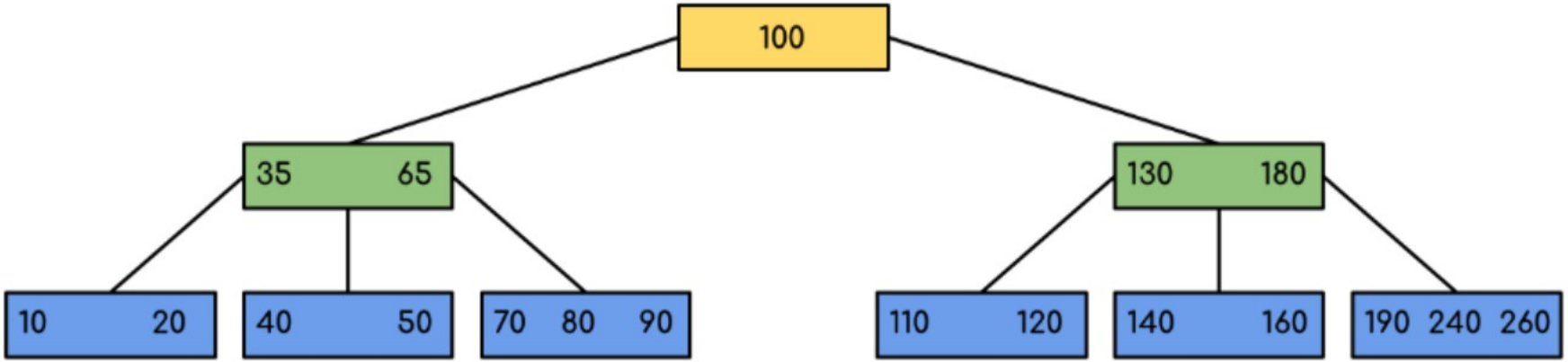
- No PostgreSQL, temos a nossa disposição vários tipos de índices, entre eles:
- B-tree
- Hash
- GiST
- GIN.

- Estes usam algoritmos diferentes que são utilizados para diferentes tipos de consultas.

B-tree

- Padrão utilizado no CREATE INDEX
- Utiliza árvore balanceada: organiza os valores em uma estrutura de árvore com vários ramos que se reequilibram automaticamente para garantir performance constante nas consultas.
- Pode operar todos os tipos de dados,
- Pode ser utilizado para recuperar valores nulos.
- Este tipo de índice é projetado para trabalhar muito bem com cache

B-tree



Hash

- É mais utilizado em bancos de dados não relacionais.
- Utilizam uma função de hash para converter um valor (ex: CPF, e-mail, código) em um código numérico fixo

```
CPF (123.456.789-00)  →  função hash →  8F23AB
```

- Na busca, em vez de comparar valor por valor, o SGBD compara os códigos hash, o que é muito mais rápido — mas só funciona para igualdade (=).

Hash

Situação	Índice Hash funciona bem?
`WHERE cpf = '123'`	✅ Excelente
`WHERE email = 'x@y.com'`	✅ Excelente
`WHERE idade BETWEEN 20 AND 30`	❌ Não funciona (sem range)
`ORDER BY nome`	❌ Não ajuda
`LIKE 'Jo%'`	❌ Não funciona

- Indices hash são especialmente úteis para buscas por igualdade em colunas com valores aleatórios ou não sequenciais, como CPFs, e-mails, UUIDs, etc

Hash

- Até versões anteriores ao PostgreSQL 10, índices hash não garantiam transações seguras, exigindo reconstrução manual em caso de falhas.

- Nas versões mais recentes, passaram a ser seguros, mas ainda são pouco utilizados, já que os índices B-Tree costumam oferecer desempenho semelhante com mais flexibilidade.

```
CREATE INDEX nome_index ON nome_tabela  USING hash (nome_coluna);

CREATE INDEX idx_clientes_email_hash ON clientes USING hash (email);

SELECT * FROM clientes WHERE email = ‘aaaa@gmail.com’;
```

GIN

- GIN (Generalized Inverted Indexes) foi projetado para lidar com casos em que os itens a serem indexados são valores compostos e as consultas a serem tratadas pelo índice precisam buscar valores de elementos que aparecem dentro dos itens compostos.

- Diferente do B-Tree que são otimizados para valores chave únicos

id	tags
1	{'promoção', 'verão'}
2	{'inverno', 'promoção'}
3	{'verão'}

GIN

- O índice GIN funciona como uma estrutura semelhante a uma árvore invertida (árvore de componentes), que armazena todos os valores possíveis encontrados dentro das colunas indexadas.

- Para cada valor encontrado, o índice mantém uma lista (ou conjunto) de ponteiros para todas as linhas da tabela onde aquele valor aparece.

- Por isso, uma única linha da tabela pode aparecer várias vezes dentro do índice,

GIN

- 'verão' → {linha 1, linha 3}

id	tags
1	{'promoção', 'verão'}
2	{'inverno', 'promoção'}
3	{'verão'}

CREATE INDEX nome_index ON nome_tabela **USING GIN** (nome_coluna);

GIN

-- habilitar extensão uma vez no banco
CREATE EXTENSION IF NOT EXISTS pg_trgm;

-- busca por pedaços do nome do produto (ex.: ILIKE '%aço%')
-- gin_trgm_ops permite criar índices GIN baseados em trigramas
CREATE INDEX idx_produtos_nome_trgm ON produtos **USING gin** (nome gin_trgm_ops);

--Busca todos os produtos cujo nome contém a palavra "aço" em qualquer posição.
--ILIKE é semelhante ao LIKE, mas ignora maiúsculas e minúsculas.
SELECT produto_id, nome **FROM** produtos **WHERE** nome **ILIKE** '%caixa%';

-- Usa o operador de similaridade (%) oferecido pela extensão pg_trgm.
--Retorna registros onde o texto em nome é similar ou se parece com 'cimento', mesmo que não seja exatamente igual.
SELECT * **FROM** produtos **WHERE** nome % 'camisa';

GIN

```
CREATE TABLE catags( id SERIAL PRIMARY KEY, tags TEXT[] -- coluna com array de texto );
INSERT INTO catags (tags) VALUES (ARRAY['promoção', 'verão']);
INSERT INTO catags (tags) VALUES (ARRAY['inverno', 'liquidação']);
INSERT INTO catags (tags) VALUES (ARRAY['infantil', 'novidade']);
INSERT INTO catags (tags) VALUES (ARRAY['esporte', 'leve', 'confortável']);
INSERT INTO catags (tags) VALUES (ARRAY['casual']);
INSERT INTO catags (tags) VALUES (ARRAY['luxo', 'exclusivo', 'premium']);
INSERT INTO catags (tags) VALUES (ARRAY['tecnologia', 'lançamento']);
INSERT INTO catags (tags) VALUES (ARRAY['ecológico', 'reciclável']);
INSERT INTO catags (tags) VALUES (ARRAY['importado', 'alta performance']);
INSERT INTO catags (tags) VALUES (ARRAY['básico', 'cotidiano’]);
CREATE INDEX idx_catags_tags_gin ON catags USING gin (tags);
--@> significa contém
-- os dois vão retornar o mesmo
SELECT * FROM catags WHERE tags @> ARRAY['promoção'];
SELECT * FROM catags WHERE 'promoção' = ANY(tags);
-- Busca itens que contenham AO MESMO TEMPO 'promoção' E 'verão'
SELECT * FROM catags WHERE tags @> ARRAY['promoção', 'verão'];
-- Busca itens que contenham 'promoção' OU 'verão'
SELECT * FROM catags WHERE 'promoção' = ANY(tags) OR 'verão' = ANY(tags);
--Verifica se existe algum elemento em comum entre os dois lados
SELECT * FROM catags WHERE tags && ARRAY['inverno', 'luxo’];11
```

GiST

- GiST é uma abreviação de “árvore de busca generalizada”.
- Esta é uma árvore de busca balanceada, assim como o b-tree
- Diferente da B-Tree, o GiST não depende de operadores de comparação tradicionais (<, >, =).

GiST

- O GiST é útil quando para os tipos de dados armazenados os operadores de comparação tradicionais simplesmente não fazem sentido, ou seja, dados que não têm ordem natural, como:

- Coordenadas geográficas (PostGIS)
- Intervalos de datas (ranges)
- Imagens

- O GiST é flexível e ideal para estes tipos de dados pois:

- Permite definir uma regra para distribuir dados de um tipo arbitrário em uma árvore balanceada
- Permite definir um método para usar essa representação para acesso por algum operador.

GiST

- Pense o seguinte, para criar um método de indexação do zero, um desenvolvedor precisaria dominar profundamente o funcionamento interno do SGBD.
- Isso inclui definir como os dados serão estruturados na memória, como serão armazenados em disco e ainda implementar mecanismos de controle de concorrência e bloqueio para garantir segurança em acessos simultâneos.
- O GiST funciona como um “template de índice”: ele cuida de toda a parte estrutural e o desenvolvedor só precisa informar como comparar os dados. Por isso ele é considerado um framework para criação de novos métodos de acesso.

```
CREATE INDEX nome_index ON nome_tabela  USING gist (nome_coluna);
```

GiST

```
CREATE TABLE reservas (
  id SERIAL,
  periodo daterange
);

INSERT INTO reservas (periodo) VALUES
(daterange('2025-01-05', '2025-01-12')),   -- sobrepõe
(daterange('2025-01-10', '2025-01-15')),   -- sobrepõe
(daterange('2025-01-01', '2025-01-08')),   -- NÃO sobrepõe
(daterange('2025-01-18', '2025-01-25')),   -- sobrepõe (final)
(daterange('2025-02-01', '2025-02-10')),   -- NÃO sobrepõe
(daterange('2025-01-19', '2025-01-21'));   -- sobrepõe (final)

CREATE INDEX idx_reservas_gist ON reservas USING gist (periodo);
```

- && significa “tem interseção?”
- Verifica se há algum elemento em comum entre os dois lados.

```
SELECT * FROM reservas WHERE periodo && daterange('2025-01-10', '2025-01-20');
```

GiST

```
-- Criar tabela de locais com coordenadas
CREATE TABLE locais (
    id SERIAL PRIMARY KEY,
    nome TEXT NOT NULL,
    coordenada POINT -- formato: (longitude, latitude)
);

-- Inserir alguns pontos de exemplo (coordenadas fictícias)
INSERT INTO locais (nome, coordenada) VALUES
('Restaurante Central',      POINT(-49.25, -16.68)),
('Supermercado Popular',     POINT(-49.23, -16.67)),
('Posto de Gasolina',        POINT(-49.30, -16.70)),
('Praça Municipal',          POINT(-49.24, -16.65)),
('Escola Estadual',          POINT(-49.20, -16.69)),
('Hospital Regional',        POINT(-49.28, -16.72));

CREATE INDEX idx_locais_gist ON locais USING gist (coordenada);

-- <-> é o operador de “distância” -> Traga os 3 locais mais próximos da coordenada (-49.26, -16.69).
SELECT nome, coordenada
FROM locais
ORDER BY coordenada <-> POINT(-49.26, -16.69)
LIMIT 3;
```

Índices em colunas múltiplas

- Um índice pode ser criado utilizando mais de uma coluna da tabela.
- B-Tree multicoluna: pode ser utilizado em consultas que envolvem qualquer subconjunto das colunas, mas funciona melhor quando a primeira coluna do índice é usada na condição de busca.
- GiST e GIN multicoluna: são mais flexíveis na combinação das colunas — a consulta pode usar qualquer uma delas e ainda aproveitar o índice.

```
CREATE INDEX nome_index ON nome_tabela  (coluna1, coluna2);

CREATE INDEX produtos_mult_index ON produtos (codigo_produto, preco);

SELECT * FROM produtos WHERE codigo_produto < 400 AND preco < 300;
```

Índices concorrentes

- Ao criar um índice de forma tradicional, o PostgreSQL bloqueia a tabela para escritas (INSERT, UPDATE, DELETE) até que o índice seja finalizado.

- Em tabelas grandes, esse processo pode demorar bastante, causando impacto em sistemas em produção.

- Para evitar essa interrupção, o PostgreSQL oferece a opção de criação concorrente de índices, por meio da cláusula CONCURRENTLY.

- Ela permite que o índice seja criado sem bloquear operações de escrita, o que torna essa abordagem ideal para ambientes que precisam continuar funcionando enquanto o índice é construído.

```
CREATE INDEX CONCURRENTLY nome_index ON tabela (coluna);
```

REINDEX

- Por fim, temos a reindexação, onde quando um índice torna-se ineficiente devido ao inchaço, a utilização da reindexação passa a ser uma opção favorável para a obtenção de um máximo desempenho dos índices

```
REINDEX TABLE nome_tabela;
```

```
CREATE INDEX produtos_mult_index ON produtos (codigo_produto, preco);
```

```
SELECT * FROM produtos WHERE codigo_produto < 400 AND preco < 300;
```

Continua na próxima aula...



UniRV
Universidade de Rio Verde

UNIVERSIDADE DE RIO VERDE (UniRV)

Bacharelado em Engenharia de Software

Fasoft
Faculdade de Engenharia de Software